

BCSE406L – NoSQL Databases

Review 1 – Problem Identification and System Design

Project Title:

Smart Hospital Information System using MongoDB and
Neo4j

Team Members:

Dadhania Nisarg Malaykumar (23BCE2364)

Madhav Sah (23BCE0868)

Arnav Dewan (23BCE0351)

1. Problem Statement

Hospitals generate and manage large amounts of data such as patient records, doctor details, appointments, prescriptions, diseases, medicines, and treatment history. This data is both flexible in structure and highly interconnected. In a traditional relational database, representing and querying these relationships may require multiple tables and complex joins. The proposed Smart Hospital Information System uses MongoDB and Neo4j to address these requirements. MongoDB will store flexible hospital records as documents, while Neo4j will represent relationships between patients, doctors, departments, diseases, and medicines as a graph. The system aims to demonstrate how multiple NoSQL database models can be combined to efficiently manage different types of healthcare data.

Problem

How can flexible hospital data and complex relationships between healthcare entities be efficiently managed using suitable NoSQL database models?

2. Objectives

1. To develop a centralized hospital information management system.
2. To store flexible hospital records using MongoDB.
3. To represent complex relationships between healthcare entities using Neo4j.
4. To provide efficient relationship-based queries for hospital data.
5. To demonstrate the advantages of using multiple NoSQL database models in one application.
6. To analyze the suitability of NoSQL databases for healthcare data management.

3. Literature Survey

3.1 Purpose

The literature survey focuses on existing research related to:

- NoSQL databases in healthcare
- MongoDB-based healthcare systems
- Electronic Health Records using NoSQL
- Graph databases in healthcare
- Neo4j-based medical systems
- Healthcare knowledge graphs
- Patient relationship analysis
- Multi-model/polyglot persistence

3.2 Literature Survey

No.	Research Area	Database/Technology	Main Contribution	Identified Limitation
1	Electronic Health Records	MongoDB/NoSQL	Explored NoSQL persistence for standardized EHR data	Mainly focused on document storage
2	EHR Query Processing	MongoDB	Demonstrated querying of standardized healthcare records	Limited relationship analysis

No.	Research Area	Database/Technology	Main Contribution	Identified Limitation
3	Healthcare Database Comparison	Neo4j & PostgreSQL	Compared graph and relational approaches for healthcare data	Did not combine both databases
4	EHR Data Modeling	NoSQL	Compared different database models for healthcare data	No complete multi-model implementation
5	Disease Knowledge Graph	Neo4j	Represented relationships between medical concepts	Focused mainly on disease knowledge
6	Disease Risk Analysis	Neo4j/Graph	Used graphs to analyze relationships between health factors	Specialized clinical use case
7	Patient Graph Analysis	Graph Database	Modeled patient-related healthcare factors	Not a general hospital management system
8	Healthcare Data Modeling	NoSQL	Proposed NoSQL approaches for semi/unstructured health data	Focused mainly on database design
9	Clinical Data Modeling	Neo4j	Represented standardized clinical data using graphs	Focused on clinical research
10	Medical Knowledge Integration	Neo4j	Connected clinical information and medical concepts	Limited hospital administration functionality

Literature Survey Observation

Existing research demonstrates that document-oriented NoSQL databases are suitable for flexible healthcare records, while graph databases are useful for highly connected medical information. However, many existing systems focus on a specific healthcare problem or a single database model.

4. Research Gaps

Based on the literature survey, the following gaps are identified:

Gap 1 – Single Database Model

Most existing approaches focus on either document-based or graph-based storage.

Proposed solution:

Combine MongoDB and Neo4j in a single application.

Gap 2 – Complex Healthcare Relationships

Relationships between patients, doctors, diseases, medicines, and departments can become complex.

Proposed solution:

Use Neo4j to explicitly represent these relationships and perform graph traversal queries.

Gap 3 – Flexible Medical Records

Different patients may have different medical information and records may evolve over time.

Proposed solution:

Use MongoDB's flexible document structure.

Gap 4 – Limited Practical Multi-Model Systems

Existing research often focuses on specific clinical analysis rather than demonstrating a complete hospital management prototype using multiple NoSQL models.

Proposed solution:

Develop a practical hospital information system using MongoDB + Neo4j.

5. Requirement Analysis

5.1 Functional Requirements

Patient Management

- Register patients.
- Update patient information.
- View patient records.
- Search patients.

Doctor Management

- Register doctors.
- Update doctor information.
- Assign doctors to departments.
- View doctor details.

Appointment Management

- Book appointments.
- View appointments.
- Update appointment status.
- Cancel appointments.

Prescription Management

- Create prescriptions.
- View prescription history.
- Associate medicines with patients.

Disease Management

- Record patient diseases.
- Associate patients with diseases.
- Associate doctors with treatments.

Graph-Based Queries

The system should support:

- Finding doctors who treated a patient.
- Finding patients treated by a doctor.
- Finding doctors in a department.
- Finding diseases associated with a patient.
- Finding medicines prescribed to a patient.
- Finding relationships between patients, doctors, diseases and departments.

5.2 Non-Functional Requirements

- Scalability: Support increasing amounts of hospital data.
- Flexibility: Accommodate changing patient and medical record structures.
- Performance: Provide efficient data and relationship queries.
- Security: Restrict access to authorized users.

- Maintainability: Keep the application modular and easy to modify.
- Availability: Maintain access to hospital information during normal operation.

6. Selection of NoSQL Databases

6.1 MongoDB

MongoDB is selected as the document database because hospital records contain flexible and semi-structured information.

MongoDB will store:

- Patients
- Doctors
- Appointments
- Prescriptions
- Medical records
- Billing information

Example MongoDB Document

```
{
  "patientId": "P101",
  "name": "Rahul",
  "age": 35,
  "bloodGroup": "B+",
  "contact": "XXXXXXXXXX",
  "medicalHistory": [
    {
      "disease": "Diabetes",
      "year": 2024
    }
  ]
}
```

6.2 Neo4j

Neo4j is selected as the graph database because hospital information contains many interconnected entities.

Neo4j will represent:

Nodes

Patient

Doctor

Department

Disease

Medicine

Relationships

Patient —TREATED_BY—> Doctor

Doctor —BELONGS_TO—> Department

Patient —DIAGNOSED_WITH—> Disease

Doctor —TREATS—> Disease

Doctor —PRESCRIBED—> Medicine

Patient —TAKES—> Medicine

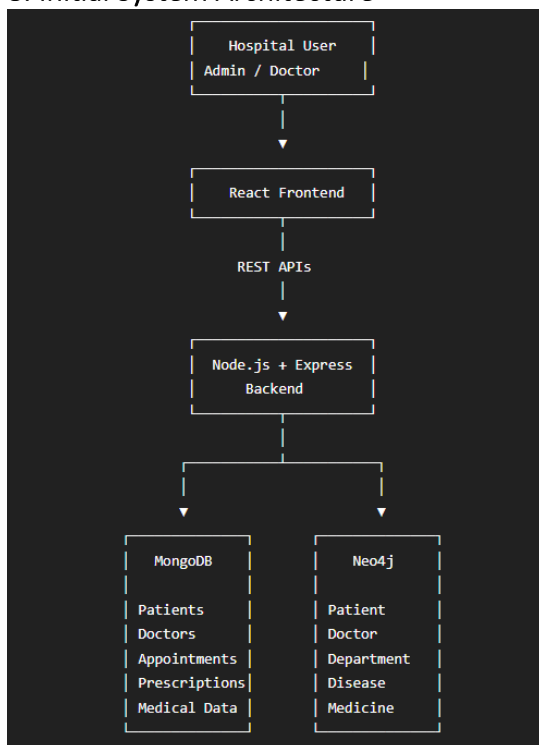
7. Why NoSQL Instead of RDBMS?

The project does not claim that NoSQL is universally better than relational databases. The reason for selecting NoSQL is based on the nature of the data.

Requirement	RDBMS	Proposed NoSQL Approach
Flexible patient records	Requires predefined schema	MongoDB supports flexible documents
Semi-structured medical data	Less flexible	MongoDB is suitable
Complex relationships	Multiple joins	Neo4j directly models relationships
Multi-hop graph queries	Can become complex	Neo4j graph traversal
Different data models	Primarily relational	MongoDB + Neo4j
Schema evolution	More rigid	MongoDB provides greater flexibility

Therefore, a combination of MongoDB and Neo4j is suitable for demonstrating different NoSQL models for different hospital workloads.

8. Initial System Architecture



Data Flow

User → React → Node.js/Express → MongoDB/Neo4j → Response → User

MongoDB handles document-based operations, while Neo4j handles relationship-based operations.

9. Database Schema

9.1 MongoDB Collections

Patient

patientId
name
age
gender
bloodGroup
contact
address
medicalHistory

Doctor

doctorId
name
specialization
departmentId
contact
experience

Appointment

appointmentId
patientId
doctorId
date
time
status

Prescription

prescriptionId
patientId
doctorId
medicines
instructions
date

9.2 Neo4j Graph Schema

Nodes

(Patient)
(Doctor)
(Department)
(Disease)
(Medicine)

Relationships

(Patient)-[:TREATED_BY]->(Doctor)
(Doctor)-[:BELONGS_TO]->(Department)

(Patient)-[:DIAGNOSED_WITH]->(Disease)
(Doctor)-[:TREATS]->(Disease)
(Doctor)-[:PRESCRIBED]->(Medicine)
(Patient)-[:TAKES]->(Medicine)

10. Project Plan

Phase	Activities
Phase 1	Problem identification and literature survey
Phase 2	Requirement analysis
Phase 3	System architecture design
Phase 4	MongoDB and Neo4j schema design
Phase 5	Backend API development
Phase 6	Frontend development
Phase 7	MongoDB–Neo4j integration
Phase 8	Testing and query evaluation
Phase 9	Performance analysis
Phase 10	Final demonstration

11. Feasibility Analysis

Technical Feasibility

The project uses MongoDB, Neo4j, React.js, Node.js, and Express.js, which are well-supported technologies with available development tools.

Economic Feasibility

The project can be developed using free/community editions and does not require specialized hardware.

Operational Feasibility

The system provides simple interfaces for common hospital operations and can be used by administrators and doctors in the prototype environment.

Implementation Feasibility

The scope is limited to core hospital management and selected graph-based queries, making it achievable within the project timeline.

12. Expected Outcome

At the end of the project, the system is expected to provide:

- A functional hospital information management prototype.
- Flexible document storage using MongoDB.
- Graph-based relationship management using Neo4j.
- CRUD operations for hospital entities.
- Relationship-based graph queries.
- Demonstration of polyglot persistence.
- Practical comparison of document and graph NoSQL models.

13. Project Limitations

- The system will use synthetic/sample hospital data.
- It is an academic prototype and not intended for real hospital deployment.
- It will not perform medical diagnosis.
- Advanced AI-based healthcare prediction is outside the current scope.
- Security will be implemented at a prototype level.

14. Future Scope

The system can later be extended with:

- AI-based medical assistance.
- Disease prediction.
- Advanced graph analytics.
- IoT-based patient monitoring.
- Real-time healthcare analytics.
- Cloud-based distributed deployment.
- Integration with Electronic Health Record systems.

15. Conclusion

The Smart Hospital Information System using MongoDB and Neo4j proposes a multi-model NoSQL approach for managing healthcare data. MongoDB will be used for flexible document-based hospital records, while Neo4j will manage complex relationships between patients, doctors, departments, diseases, and medicines. The project demonstrates how different NoSQL database models can be combined using polyglot persistence to address different data and query requirements.